UD 3 · Computación en la Nube

Conceptos, modelos, plataformas y despliegue de aplicaciones

1. FUNDAMENTOS DE LA COMPUTACIÓN EN LA NUBE

¿Qué es la computación en la nube?

- Modelo de consumo de recursos informáticos a través de Internet
- Ofrece
  - Acceso bajo demanda a servidores, almacenamiento, redes y software
  - Escalabilidad, elasticidad automática
  - Pago por uso: solo se paga lo que se consume reduce costes, aumenta eficiencia, mejora capacidad de adaptación
  - Sustituye o complementa infraestructuras on-premise (alojamiento en la empresa de los propios servidores, sistemas y aplicaciones físicamente en sus instalaciones o centro de datos)

Por qué surge la nube

- Limitaciones del modelo on-premise
- Escalado lento y costoso
- Infraestructura infrautilizada
- Necesidad de agilidad empresarial

Evolución del modelo tradicional

- Modelo clásico: compra y mantenimiento de servidores propios
- Problemas: costes elevados, infrautilización, escalado lento
- La nube introduce flexibilidad y rapidez
- Permite adaptarse a cambios de carga y negocio

Cómo funciona el Cloud Computing

El funcionamiento del cloud computing se basa en:

- Recursos virtuales compartidos (hardware, redes, almacenamiento, software) alojado en centros de datos remotos.
- Acceso a través de una red pública (normalmente Internet) o privada.
- Un sistema de gestión centralizado que coordina el acceso, el uso y la seguridad de los servicios.
- Mecanismos de virtualización y automatización para asignar dinámicamente recursos según la demanda.
- Los usuarios acceden a estos servicios mediante interfaces web, APIs, herramientas, etc. sin necesidad de gestionar la infraestructura física subyacente.
- El Cloud Computing representa una evolución clave en la forma de consumir tecnología. Permite a las organizaciones ser más ágiles, eficientes y competitivas, adaptándose a un entorno digital que cambia con rapidez

Modelos de despliegue

- Nube pública
  - Infraestructura propiedad de un proveedor externo (como Google Cloud, AWS o Azure).
  - Recursos compartidos entre múltiples clientes.
  - Acceso por Internet.
  - Ejemplo: Google Cloud Platform.

- Nube privada
  - Infraestructura usada exclusivamente por una organización.
  - Puede estar alojada en sus propias instalaciones (on-premise) o en un proveedor.
  - Mayor control y seguridad.

Modelos de despliegue

- Nube híbrida
  - Combina nubes públicas y privadas.
  - Permite mover datos o cargas de trabajo entre ambas según necesidades.
  - Favorece el cumplimiento normativo y la flexibilidad operativa

- Multi-cloud: uso de varios proveedores

Ventajas del Cloud computing

Ventaja	Descripción
Escalabilidad	Recursos que se adaptan al crecimiento o a picos de demanda automáticamente.
Acceso global	Uso de servicios desde cualquier lugar con conexión a Internet.
Rapidez	Despliegue ágil de nuevos entornos o aplicaciones.
Ahorro de costes	No requiere inversiones en hardware ni mantenimiento físico.
Seguridad	Los proveedores aplican medidas avanzadas de protección y cumplen normativas internacionales.
Innovación continua	Acceso a tecnologías emergentes (IA, Big Data, IoT) sin costes de implementación elevados.

Desventajas del Cloud computing

Desventaja	Descripción
Dependencia del proveedor	Puede dificultar cambiar de plataforma
Privacidad	Se deben considerar las normativas como RGPD (Reglamento General de Protección de Datos.
Gestión de costes	Aunque el pago por uso es ventajoso, una mala planificación puede generar gastos innecesarios.
Formación de personal	Los entornos cloud requieren nuevas habilidades (cloud, DevOps, IaC...).

2. MODELOS DE SERVICIO

Modelos de servicio en la nube

- Los modelos de servicio en la nube siguen el principio del *"as-a-Service"*: el proveedor en la nube suministra recursos bajo demanda a través de Internet, eliminando la necesidad de gestionar infraestructura física en local (*on-premise*).

Modelos de servicio

- IaaS: Infraestructura como Servicio
- PaaS: Plataforma como Servicio
- CaaS: Contenedores como Servicio
- SaaS: Software como Servicio

Modelos de servicio en la nube - IaaS

- El proveedor se encarga del hardware y la infraestructura subyacente : servidores (máquinas) virtuales, redes, almacenamiento.

- El usuario gestiona el sistema operativo, datos y aplicaciones

- Uso habitual:
  - Migración de servidores tradicionales
  - Montaje de entornos de pruebas/desarrollo
  - Escenarios con necesidad de control granular

- Ejemplos:
  - Amazon EC2 (AWS)
  - Azure Virtual Machines
  - Google Compute Engine

Modelos de servicio en la nube - IaaS

- Ventajas:
  - Control total del entorno
  - Escalabilidad inmediata
  - Integración con sistemas heredados
  - Facturación por consumo real

- Desventajas:
  - Mayor responsabilidad técnica
  - Gestión de parches, actualizaciones y seguridad a cargo del cliente

Modelos de servicio en la nube - CaaS

- El proveedor ofrece una plataforma para ejecutar y gestionar contenedores, normalmente basada en Docker y Kubernetes
- Proveedor gestiona hw, red, sevidores, so, cúster de kubernetes, escalado y alta disponibilidad del clúster
- El usuario gestiona imágnes Docker, contendores, despliegues, datos y configuración de la aplicación
- Ideal para microservicios
- Ejemplos: Amazon EKS, Azure AKS, Google GKE

Modelos de servicio en la nube - CaaS

- Uso habitual
  - Microservicios y aplicaciones distribuidas
  - DevOps con ciclos de vida rápidos
  - Portabilidad entre nubes y entornos híbridos

- Ventajas:
  - Mayor portabilidad (multinube o híbrido)
  - Despliegue ágil y automatización
  - Ideal para arquitecturas modernas

- Inconvenientes:
  - Dependencia del ecosistema Kubernetes
  - Riesgos asociados a la seguridad de contenedores (comparten el kernel)

Modelos de servicio en la nube - PaaS

- El proveedor ofrece una plataforma para desarrollar, ejecutar y desplegar aplicaciones, sin que el usuario tenga que gestionar la infraestructura.

- El proveedor gestiona hw, servidores, red, SO y escalabilidad

- El usuario (desarrollador) se centra en el código, lógica de negocio, datos, configuración de la app.

- Uso habitual
  - Aplicaciones web personalizadas
  - Backends de apps móviles
  - Entornos de desarrollo ágil

- Ejemplos: Google App Engine, Azure App Service, AWS Elastik Beanstalk, Heroku

Modelos de servicio en la nube - PaaS

Ventajas

- Enfoque en el desarrollo
- Despliegue rápido
  - Crear y publicar una aplicación lleva minutos
  - Ideal para prototipos y proyectos ágiles
- Escalabilidad automática
  - La plataforma escala la aplicación según la demanda
  - No es necesario dimensionar infraestructura manualmente
- Menor carga de administración
- Muy alineado con metodologías DevOps
  - CI/CD, monitorización, logs, bases de datos gestionadas
- Alta disponibilidad
  - La plataforma suele incluir balanceo y redundancia por defecto

Modelos de servicio en la nube - PaaS

Desventajas

- Menor control del entorno
  - No se puede elegir SO y configuraciones avanzadas
- Dependencia del proveedor
  - APIs y servicios suelen ser específicos de la plataforma
- Limitaciones técnicas
  - Restricciones en lenguajes soportados
- Coste elevado a largo plazo
- Responsabilidad compartida
  - Proveedor gestiona la plataforma
  - Cliente responsable de datos, código, seguridad de la aplicaci´ón

Modelos de servicio en la nube - SaaS

- Se ofrece una aplicación lista para usar, completamente gestionada por el proveedor, accesible generalmente a través del navegador.

- El usuario no instala ni mantiene nada.

- Ejemplos
  - Google Workspace
  - Microsoft 365
  - Salesforce

- Uso habitual
  - Correo electrónico
  - Herramientas de productividad
  - CRM y ERP online
  - Aplicaciones colaborativas

Modelos de servicio en la nube - SaaS

- Ventajas
  - No requiere instalación ni mantenimiento
  - Rápido despliegue y puesta en marcha
  - Escalabilidad automática
  - Siempre actualizado

- Desventajas
  - Cero control sobre la infraestructura
  - Personalización muy limitada
  - Posibles problemas de integración

Modelos de servicio en la nube - Comparativa

Elemento	On-Premises	IaaS	CaaS	PaaS	SaaS
Datos y contenido	✅	✅	✅	✅	✅
Aplicación	✅	✅	✅	✅	❌
Entorno de ejecución	✅	✅	✅	❌	❌
Sistema operativo	✅	✅	❌	❌	❌
Virtualización/Contenedores	✅	✅	❌*	❌	❌
Hardware, red, almacenamiento	✅	❌	❌	❌	❌

* En CaaS la virtualización se abstrae completamente del cliente.

Modelos de servicio en la nube - Analogía

Modelo	Analogía habitacional
On-Premises	Construir una casa desde cero: compras el terreno, materiales y la construyes tú.
IaaS	Alquilar una parcela con cimientos y servicios básicos: tú edificas.
CaaS	Alquilar una vivienda donde puedes traer tus propios muebles (contenedores).
PaaS	Alquilar una casa ya amueblada: solo llevas tus cosas personales (tu código).
SaaS	Hospedarte en un hotel con todo incluido: todo gestionado por el proveedor.

Modelos de servicio en la nube - Conclusión

- La elección entre IaaS, PaaS, SaaS y CaaS depende de:
  - el tipo de proyecto
  - el nivel de control deseado,
  - los recursos disponibles y
  - la experiencia del equipo.

- En la práctica, muchas organizaciones adoptan modelos híbridos o combinados, integrando varios servicios para cubrir diferentes necesidades

3. PLATAFORMAS CLOUD

Plataformas de servicios en la nube

- Principales plataformas

Proveedor	Cuota Mercado (2023)	Año Lanzamiento	Foco Principal
AWS	31%	2006	Escalabilidad global y mayor catálogo
Azure	24%	2010	Integración Microsoft e híbrida
GCP	11%	2011	IA/ML y análisis de datos

- Otras plataformas

OVHCloud, Alibaba Cloud…

Plataformas de servicios en la nube - AWS

Amazon Web Services (AWS)

- Proveedor más grande y consolidado.
- Soluciones que incluyen computación, almacenamiento, bases de datos, inteligencia artificial y machine learning, entre otros.
- Más de 200 servicios.
- Infraestructura distribuida en múltiples regiones y zonas de disponibilidad, garantizando alta redundancia y bajas latencias, lo que lo convierte en la opción preferida para aplicaciones críticas.
- Complejidad para nuevos usuarios.
- Costes ocultos sin gestión adecuada
- Documentación extensa pero abrumadora

Plataformas de servicios en la nube – Microsoft Azure

Azure

- Integración perfecta con herramientas empresariales como Windows Server, Active Directory y Office 365.
- Ofrece servicios de IaaS, PaaS y soluciones de nube híbrida.
- Data centers en más de 60 regiones.
- Fuerte enfoque en seguridad y cumplimiento normativo popular en sectores regulados como banca, gobierno y salud.
- Soporte para tecnologías como inteligencia artificial, IoT y blockchain.
- Herramientas de desarrollo (Azure DevOps, VS Code).
- Menor diversidad vs AWS.
- Curva de aprendizaje para no-usuarios Microsoft.
- Costes menos transparentes en algunos servicios.

Plataformas de servicios en la nube – Google Cloud

Google Cloud

- Amplio portafolio que incluye computación, almacenamiento, big data, machine learning y herramientas de análisis.
- Data centers de alta eficiencia con rendimiento excepcional y baja latencia.
- Reconocido por sus capacidades en análisis de datos (BigQuery) e inteligencia artificial (TensorFlow y Vertex AI).
- Óptimo para escalabilidad, innovación y soluciones basadas en datos.
- Precios competitivos (descuentos por uso sostenido).
- Kubernetes nativo (GKE, Anthos).
- Menor cobertura geográfica.
- Ecosistema menos maduro para legacy systems.
- Menor adopción en empresas tradicionales.

Plataformas de servicios en la nube – Terminología

Componentes y terminología usada

	AWS	Azure	Google cloud
Máquinas virtuales	EC2	Azure Vms	Compute engine
Discos persistentes	EBS (Elastic Block Store)	Disk Storage	Persistent Disk
Almacenamiento de objetos	S3	Blob Storage	Cloud Storage
Archivos compartidos	EFS	Azure Files	Filestore
Redes virtuales	VPC	VNet	VPC
CDN	CloudFront	Akamai	Cloud CDN
Bases de datos	RDS	Azure SQL	Cloud SQL
Contendores	ECS EKS (Kubernetes)	AKS (Az. Kuber. Service) Azure Container Instances	GKE (Google Kubernetes Engine)
CI/CD	CodePipeline	Azure DevOps GitHub Actions	Cloud Build
Ejecución de código bajo demanda (evento)	Lamda	Azure Functions	Cloud Functions

Plataformas de servicios en la nube – Terminología

Casos reales

- Net**flix**

  Usa S3 para las películas, Lambda para recomendaciones y RDS para guardar tu perfil

- Airbnb

  Las fotos están en S3+CloudFront, y cuando buscas un piso, se activan funciones Lambda

Plataformas de servicios en la nube – Comparativa

Criterio	AWS	Azure	GCP
Mejor para	Escalabilidad masiva	Entornos Microsoft	IA/ML y datos
Precios	Flexibles pero complejos	Paquetes empresariales	Los más competitivos
Global Reach	30+ regiones	60+ regiones	35+ regiones
Ejemplo	Netflix, Airbnb	BMW, Walmart	Spotify, Twitter

- Recomendaciones Clave
  - Para startups tecnológicas: GCP por costos y herramientas de IA
  - Corporaciones Microsoft: Azure para integración perfecta
  - Proyectos globales complejos: AWS por su madurez y escalabilidad
  - Multi-nube: Considera combinar AWS/GCP para IA o Azure/AWS para híbrido
  - "No existe un proveedor único ideal, sino el que mejor se alinee con tus cargas de trabajo actuales y futuras"

Plataformas de servicios en la nube – Casos de éxito

- Airbnb (AWS)
  - Desafío: Manejar picos de tráfico masivos (ej. Año Nuevo).
  - Solución: AWS Auto Scaling y Amazon RDS para escalar automáticamente.
  - Resultado:
    - Soporta 150 millones de usuarios con 99.99% disponibilidad.
    - Reduce costes en un 40% vs. infraestructura física.

- BMW (Azure)
  - Desafío: Digitalizar fábricas y conectar millones de sensores IoT.
  - Solución: Usó Azure IoT Hub y AI para análisis en tiempo real.
  - Resultado:
    - 30% más eficiencia en producción.
    - Menos fallos en la línea de ensamblaje gracias a mantenimiento predictivo.

Plataformas de servicios en la nube – Casos de éxito

- Spotify (GCP)
  - Desafío: Procesar 600 TB de datos diarios de usuarios.
  - Solución: Migró a Google BigQuery y Dataflow.
  - Resultado:
    - Análisis de datos 100x más rápido.
    - Recomendaciones personalizadas en tiempo real.

- NASA (AWS)
  - Proyecto: Almacenar y procesar imágenes del telescopio James Webb.
  - Solución: Amazon S3 y machine learning para clasificar datos astronómicos.
  - Impacto:
    - Científicos acceden a 10 TB diarios de imágenes sin retardo.

- Mercado Libre (Multi-nube: AWS + GCP)
  - Estrategia: Usa AWS para e-commerce y GCP para big data.
  - Resultado:
    - Soporta 2.5 millones de pedidos por hora en días como el CyberMonday.

Plataformas de servicios en la nube – Curiosidades

- AWS nació por necesidad interna de Amazon al tener exceso de capacidad en sus servidores. Hoy genera más ganancias que todo el negocio e-commerce de Amazon.

- Netflix migró a AWS después de que sus servidores físicos dejaran de funcionar 3 días (2008) → implantaron AWS para lograr escalabilidad y resiliencia. Hoy, el 100% de su streaming corre en la nube.

- Google Cloud usa la misma infraestructura que YouTube → garantiza baja latencia y alto rendimiento.

- Xbox Cloud Gaming (Microsoft) ejecuta los juegos en la nube de Xbox (xCloud) en servidores Azure sin necesidad de consolas físicas.

- El primer servicio en la nube fue el correo electrónico, siendo Hotmail (1996) y luego Gmail (2004) los primeros ejemplos de SaaS, aunque no se llamaba "cloud" todavía.

4. INFRAESTRUCTURA COMO CÓDIGO

4.1 Infraestructura como código (IaC)

- La gestión eficiente de la infraestructura es fundamental para el desarrollo ágil y escalable de aplicaciones.
- Tienes un rol fundamental en DevOps, donde impulsa la automatización y la colaboración entre equipos.
- Es la capacidad de aprovisionar y administrar infraestructura de computación (sistemas operativos, conexiones a bases de datos, recursos de almacenamiento, redes y más) mediante código, en lugar de utilizar procesos manuales de configuración.
- La gestión manual de la infraestructura es un proceso lento, propenso a errores humanos y difícil de escalar.
- IaC resuelve estos problemas definiendo como queremos que sea la infraestructura con archivos de configuración.

4.2 Infraestructura como código (IaC) - Beneficios

Beneficios

- Replicación de entornos es posible replicar exactamente el mismo entorno (servidores, redes, configuraciones personalizadas) en diferentes ubicaciones (una nueva sucursal, por ejemplo) o sistemas usando los mismos archivos de configuración, implementando un entorno idéntico.

- Reducir riesgos asociados a configuraciones erróneas
  - Las configuraciones se definen una vez y se aplican consistentemente

- Fácil de integrar con control de versiones:
  - Permite un control de versiones de la infraestructura
  - Si ocurre un error, es posible volver a una versión anterior estable del código de infraestructura
  - Gestionar diferentes versiones de la infraestructura en paralelo

4.2. Infraestructura como código (IaC) - Beneficios

Beneficios

- Adaptar la infraestructura escalando bajo demanda
- Controlar costes al eliminar tareas redundantes

4.3. Infraestructura como código (IaC) - ¿Cómo funciona?

- El código de infraestructura describe mediante archivos la arquitectura de un sistema, incluyendo:
  - Servidores físicos o virtuales
  - Configuraciones de red
  - Sistemas operativos y sus configuraciones
  - Recursos de almacenamiento
  - Reglas de seguridad y acceso

- Las herramientas de IaC tratan esos como código fuente:
  - Se escriben en IDEs con soporte para sintaxis y validación
  - Se almacenan en sistemas de control de versiones
  - Siguen prácticas de desarrollo como revisiones de código
  - Pueden incluirse como parte del código principal de una aplicación

4.4. Infraestructura como código (IaC) – Enfoques de implementación

- Declarativo -> se describe el estado final de la infraestructura (sus recursos y configuraciones), sin especificar los pasos para conseguirlo.
  - Más fácil de escribir y mantener
  - Ideal para la mayoría de casos de uso estándar
  - Ejemplos: Ansible*, Terraform, AWS CloudFormation, Puppet
  - Formatos: YAML, JSON, HCL (Terraform)

- Imperativo -> se epsecifica cada pasos que hay que dar para construir la infraestructura.
  - Mayor control sobre el proceso
  - Necesario para configuraciones complejas donde el orden de ejecución es crítico
  - Ejemplo: scripts de bash, Chef, Ansible*

* Usa los dos enfoques: permite usar YAML con lógica imperativa (bucles, condicionales)

4.4. Infraestructura como código (IaC) – Enfoques de implementación

- Ejemplo en Ansible

```yaml
# playbook.yml
- hosts: servidores
  tasks:
    - name: Instalar Apache
      apt:
        name: apache2
        state: present
    - name: Iniciar servicio
      service:
        name: apache2
        state: started
        enabled: yes
```

- Ejemplo de Terraform

```
# main.tf
resource "aws_instance" "mi_servidor_web" {
  ami           = "ami-0c55b159cbfafe1f0"  # Imagen de Ubuntu
  instance_type = "t2.micro"               # Tipo de instancia gratis
  tags = {
    Name = "ServidorWebBásico"
  }
}
```

4.4. Infraestructura como código (IaC) – IaC en DevOps

- DevOps busca integrar el desarrollo de sw con las operaciones de TI, acortando ciclos de desarrollo y permitiendo entregas continuas de software de alta calidad.

En este contexto, IaC juega un papel esencial, ya que:

- Automatiza completamente la gestión de infraestructura
  - Los cambios en infraestructura pueden implementarse mediante pipelines CI/CD
  - Permite implementar entornos completos (desarrollo y producción) con comandos
- Garantizar consistencia absoluta entre entornos de desarrollo y producción
- Integración con proveedores cloud:
  - Escalado automático basado en demanda
  - Implementación multi-nube consistente
  - Gestión centralizada de costos y recursos
- Lenguaje común para Devs y Ops:
  - Fomenta la colaboración mediante revisiones de código de infraestructura
  - Documentación siempre actualizada (el código es la documentación)

5. CI / CD EN LA NUBE

5. Integración de CI/CD en la nube

- El proceso a la hora de desplegar una aplicación sería el siguiente:

1. CI

   1. Desarrollador hace git push y dispara pipeline de CI
      - Compila el código
      - Ejecuta pruebas unitarias y de integración
      - Escanea vulnerabilidades

   2. Si todo va bien, fusiona (merge) le código en la rama principal

```yaml
# .github/workflows/ci.yml
name: CI Pipeline
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install && npm test  # Ejecuta pruebas
```

5. Integración de CI/CD en la nube

2. CD

1. Tras el CI exitoso, el pipeline de CD
   - Empaqueta el código fuente (contenedor Docker, por ejemplo)
   - Despliega en la nube
   - Ejecuta pruebas E2E (validación del funcionamiento de la aplicación de principio a fin)

2. Si todo va bien, la actualización de publica sin intervención manual

```yaml
# .github/workflows/cd.yml
name: CD to AWS
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t my-app . && docker push my-ecr-repo
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_KEY }}
      - run: aws ecs update-service --cluster my-cluster --service my-service --force-new-deployment
```

6. ORQUESTACIÓN DE CONTENEDORES. KUBERNETES

6.1. Orquestación

- Las aplicaciones actuales suelen estar compuestas por decenas, cientos o incluso miles de contenedores.

- La nube permite distribuir y gestionar estos contenedores dinámicamente a través de múltiples servidores o instancias, funcionando como si estuvieran en un mismo entorno.

- Para que puedan comunicarse y trabajar entre sí se necesitar la orquestación:

  - Automatiza el despliegue, escalado y gestión de contenedores.
  - Distribuyen cargas en clústeres.
  - Manejan fallos, reiniciando o reemplazando el contenedor que falla sin intervención humana.
  - Balancean la carga.
  - Monitorizan el estado de los contenedores y registran eventos.

6.2. Kubernetes

- Kubernetes (K8s) es una plataforma de código abierto para automatizar la implementación, el escalado y la administración de aplicaciones en contenedores.

- Fue desarrollado originalmente por Google.

6.2. Kubernetes

- Nodo: máquina física o virtual que forma parte de un cluster de Kubernetes

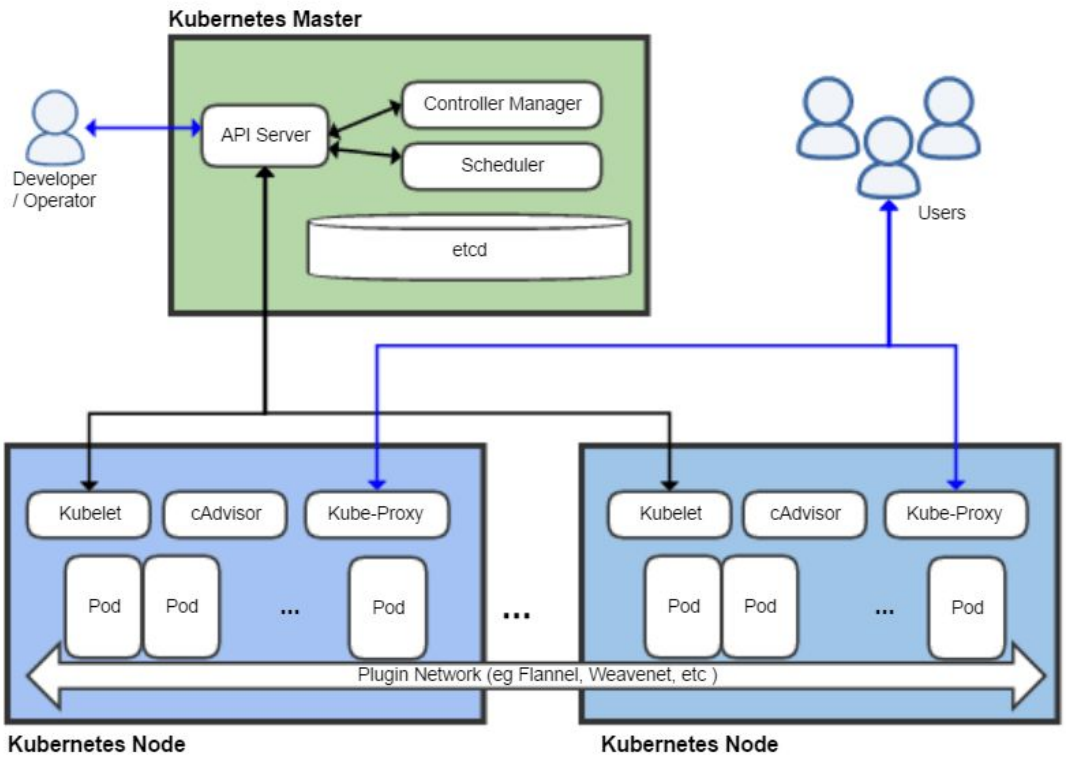
- Pod
  - Unidad más pequeña en Kubernetes
  - Suele coincidir con un contendor (aunque puede contener más de uno)

- Deployment
  - Crea y gestiona los pods
  - Escala, actualiza y repone los pods si alguno falla

- Service
  - Permite el acceso a los pods ofreciéndoles ip fija y DNS dentro del cluster
  - Balancea la carga

```
[ Usuario / Otro Pod ]
          |
          ▼
   [ Service ]  <-- IP fija, balanceo de carga
      /   |   \\
     ▼    ▼    ▼
 [Pod1] [Pod2] [Pod3]  <-- Contienen la app
     \\    |    /
      [ Deployment ] <-- Controla que siempre haya 3 pods
```

6.2. Kubernetes



6.3. Kubernetes vs Compose

Característica	Docker Compose	Kubernetes
Configuración	Archivo `docker-compose.yml`	Múltiples archivos YAML (`Deployment`, `Service`, etc.)
Escalado automático	Manual (`docker-compose up --scale`)	Automático (Horizontal Pod Autoscaler)
Autorreparación	No incluido	Reinicio automático de contenedores fallidos
Rolling updates	Limitado	Soportado nativamente con `rollingUpdate`
Exposición externa	Solo con `ports` o reverse proxy	Servicios con tipo `LoadBalancer` o `Ingress`
Monitorización integrada	No	Integrado con Prometheus, Grafana, etc.
Networking	Redes internas simples	Networking avanzado (DNS interno, políticas de red)
Entornos de producción	No recomendado	Estándar de la industria para entornos productivos
Compatibilidad multi-nube	Limitada	Alta (AWS, Azure, GCP, etc.)

- Docker Compose es ideal para entornos de desarrollo o pruebas locales por su simplicidad y rapidez.

- Kubernetes, en cambio, es la solución robusta y escalable para entornos productivos en la nube, permitiendo una gestión avanzada y resiliente de contenedores en clústeres distribuidos.

6.4. Ejemplo sencillo kubernetes

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mi-app
spec:
  replicas: 2                # ✅ Quiero 2 copias de mi aplicación
  selector:
    matchLabels:
      app: mi-app
  template:
    metadata:
      labels:
        app: mi-app
    spec:
      containers:
        - name: mi-contenedor
          image: nginx:latest  # ✅ Usa la imagen de NGINX
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: mi-app-service
spec:
  selector:
    app: mi-app              # ✅ Busca los pods con esta etiqueta
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP            # ✅ Accesible solo dentro del clúster
```

- **Deployment**:
  Crea 2 Pods con NGINX.
  Se asegura de que siempre haya esas 2 copias funcionando.
  Si un Pod falla, se reemplaza automáticamente.

- **Service**:
  Encuentra todos los Pods creados por el Deployment mediante etiquetas.
  Proporciona IP y nombre DNS.
  Reparte el tráfico entre los Pods (balanceo de carga interno).